



دانشگاه صنعتی امیرکبیر
(پلی تکنیک تهران)
دانشکده مهندسی کامپیوتر

پایان نامه کارشناسی

طراحی سامانه ثبت وقایع برای تحلیل بی درنگ کارایی فرایندهای JVM.

نگارش

محمد امین صدرنشین

استاد راهنما

دکتر حامد فریه

استاد مشاور

دکتر سید احمد جوادی

اسفند ۱۴۰۴

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

صفحه فرم ارزیابی و تصویب پایان نامه - فرم تأیید اعضاء کمیته دفاع

در این صفحه فرم دفاع یا تأیید و تصویب پایان نامه موسوم به فرم کمیته دفاع - موجود در پرونده آموزشی - را قرار دهید.

نکات مهم:

- نگارش پایان نامه/رساله باید به **زبان فارسی** و بر اساس آخرین نسخه دستورالعمل و راهنمای تدوین پایان نامه های دانشگاه صنعتی امیرکبیر باشد.(دستورالعمل و راهنمای حاضر)
- رنگ جلد پایان نامه/رساله چاپی کارشناسی، کارشناسی ارشد و دکترا باید به ترتیب مشکی، طوسی و سفید رنگ باشد.
- چاپ و صحافی پایان نامه/رساله بصورت **پشت و رو(دورو)** بلامانع است و انجام آن توصیه می شود.



دانشگاه صنعتی امیرکبیر
(پلی تکنیک تهران)

به نام خدا

تعهدنامه اصالت اثر

تاریخ: اسفند ۱۴۰۴

اینجانب **محمد امین صدرنشین** متعهد می‌شوم که مطالب مندرج در این پایان‌نامه حاصل کار پژوهشی اینجانب تحت نظارت و راهنمایی اساتید دانشگاه صنعتی امیرکبیر بوده و به دستاوردهای دیگران که در این پژوهش از آنها استفاده شده است مطابق مقررات و روال متعارف ارجاع و در فهرست منابع و مآخذ ذکر گردیده است. این پایان‌نامه قبلاً برای احراز هیچ مدرک هم‌سطح یا بالاتر ارائه نگردیده است.

در صورت اثبات تخلف در هر زمان، مدرک تحصیلی صادر شده توسط دانشگاه از درجه اعتبار ساقط بوده و دانشگاه حق پیگیری قانونی خواهد داشت.

کلیه نتایج و حقوق حاصل از این پایان‌نامه متعلق به دانشگاه صنعتی امیرکبیر می‌باشد. هرگونه استفاده از نتایج علمی و عملی، واگذاری اطلاعات به دیگران یا چاپ و تکثیر، نسخه‌برداری، ترجمه و اقتباس از این پایان‌نامه بدون موافقت کتبی دانشگاه صنعتی امیرکبیر ممنوع است. نقل مطالب با ذکر مآخذ بلامانع است.

محمد امین صدرنشین

امضا

نویسنده پایان نامه، در صورت تمایل میتواند برای پاسخگویی پایان نامه خود را به شخص
یا اشخاص و یا ارگان خاصی تقدیم نماید.

سپاسگزاری

نویسنده پایان نامه می تواند مراتب امتنان خود را نسبت به استاد راهنما و استاد مشاور و یا دیگر افرادی که طی انجام پایان نامه به نحوی او را یاری و یا با او همکاری نموده اند ابراز دارد.

محمد امین صدر نشین
اسفند ۱۴۰۴

چکیده

افزایش پیچیدگی سامانه‌های نرم‌افزاری و اجرای پیوسته آن‌ها در محیط‌های توزیع‌شده، نیاز به پایش دقیق، کم‌هزینه و قابل تحلیل را پررنگ کرده است. در این پایان‌نامه سامانه‌ای برای جمع‌آوری، جمع‌آوری و تحلیل رخدادهای عملکردی برنامه‌های مبتنی بر ماشین مجازی جاوا طراحی و پیاده‌سازی شده است. سامانه پیشنهادی از Java Flight Recorder برای دریافت رخدادهای سطح برنامه، از eBPF برای مشاهده رخدادهای سطح هسته، از Kafka برای انتقال جریان داده، از Kafka Streams برای جمع‌پنجره‌ای، و از یک بخش تحلیلی پایتونی برای تشخیص ناهنجاری‌ها استفاده می‌کند. معماری سامانه به صورت خط لوله‌ای طراحی شده است تا دریافت رخداد، جمع‌آوری، تحلیل و تولید گزارش از یکدیگر جدا باشند. در بخش دریافت، رخدادهای JFR و رخدادهای هسته به پیام‌های ساخت‌یافته تبدیل می‌شوند. سپس رخدادها در بازه‌های زمانی قابل تنظیم جمع‌آوری شده و معیارهایی مانند مصرف پردازنده، حافظه، ورودی/خروجی فایل، شبکه، قفل‌ها و شمارنده‌های سیستمی استخراج می‌شود. در مرحله تحلیل، آشکارسازهای مبتنی بر آستانه، خط پایه غلتان و روند افزایشی، گزارش ناهنجاری‌ها را تولید می‌کنند. برای ارزیابی، مجموعه‌ای از سناریوهای کنترل‌شده شامل جهش پردازنده، فشار حافظه و جمع‌آوری زباله، انفجار ورودی/خروجی فایل، ازدحام قفل، ترافیک شبکه و دسترسی حافظه‌نگاشته به فایل در مخزن پروژه پیاده‌سازی شده است. نتایج نشان می‌دهد که جداسازی اجزای خط لوله، توسعه‌پذیری سامانه را افزایش داده و امکان تحلیل همزمان رخدادهای سطح برنامه و سطح سیستم‌عامل را فراهم می‌کند.

واژه‌های کلیدی:

پایش عملکرد، تحلیل بی‌درنگ، ثبت وقایع، JVM، Java Flight Recorder

فهرست مطالب

صفحه

عنوان

۱	فهرست نمادها و کوتاه‌نوشت‌ها	۱
۲	۱ کلیات پژوهش	۲
۳	۱-۱ مقدمه	۳
۳	۲-۱ بیان مسئله	۳
۳	۳-۱ اهمیت و ضرورت	۳
۴	۴-۱ اهداف پژوهش	۴
۴	۵-۱ پرسش‌های پژوهش	۴
۴	۶-۱ ساختار پایان‌نامه	۴
۵	۲ مبانی نظری و فناوری‌های مورد استفاده	۵
۶	۱-۲ پایش عملکرد و مشاهده‌پذیری	۶
۶	۲-۲ Java Flight Recorder	۶
۶	۳-۲ eBPF و رخدادهای سطح هسته	۶
۶	۴-۲ Kafka و پردازش جریانی	۶
۷	۵-۲ تشخیص ناهنجاری	۷
۷	۶-۲ جمع‌بندی	۷
۸	۳ معماری سامانه	۸
۹	۱-۳ مقدمه	۹
۹	۲-۳ نمای کلی معماری	۹
۱۰	۳-۳ مسیرهای داده در سامانه	۱۰
۱۰	۱-۳-۳ مسیر رخدادهای JFR	۱۰
۱۰	۲-۳-۳ مسیر رخدادهای هسته	۱۰
۱۱	۴-۳ لایه جمع‌آوری داده	۱۱
۱۱	۵-۳ لایه انتقال و ذخیره‌سازی جریان داده	۱۱
۱۱	۶-۳ لایه تجمیع	۱۱

۱۲	۷-۳ لایه تحلیل
۱۳	۸-۳ لایه ارزیابی
۱۳	۹-۳ مدل داده مشترک
۱۳	۱۰-۳ دلایل انتخاب معماری لایه‌ای
۱۴	۱۱-۳ جمع‌بندی
۱۵	۴ پیاده‌سازی سامانه
۱۶	۱-۴ ساختار مخزن
۱۶	۲-۴ دریافت رخدادهای JFR
۱۶	۳-۴ دریافت رخدادهای هسته
۱۶	۴-۴ تجمیع رخدادها
۱۷	۵-۴ تحلیل و گزارش‌گیری
۱۷	۶-۴ پایش خود سامانه
۱۷	۷-۴ جمع‌بندی
۱۸	۵ ارزیابی، نتیجه‌گیری و کارهای آینده
۱۹	۱-۵ روش ارزیابی
۱۹	۲-۵ سناریوهای آزمایش
۱۹	۳-۵ معیارهای ارزیابی
۲۰	۴-۵ بحث
۲۰	۵-۵ نتیجه‌گیری
۲۰	۶-۵ کارهای آینده
۲۲	کتاب‌نامه
۲۳	آ راهنمای اجرای پروژه
۲۴	۱-آ ساخت ماژول‌های جاوا
۲۴	۲-آ اجرای اجزای اصلی
۲۴	۳-آ اجرای ارزیابی
۲۴	۴-آ خروجی‌ها

۲۶	 واژه‌نامه‌ی فارسی به انگلیسی
۲۸	 واژه‌نامه‌ی انگلیسی به فارسی

فهرست تصاویر

صفحه

شکل

۱-۳ نمای کلی معماری لایه‌ای سامانه ۹

فهرست جداول

صفحه

جدول

فهرست نمادها و کوتاه‌نوشت‌ها

JFR	Java Flight Recorder، ابزار ثبت رخدادهای زمان اجرای جاوا
JVM	Java Virtual Machine، ماشین مجازی جاوا
eBPF	extended Berkeley Packet Filter، چارچوب اجرای برنامه‌های کنترل‌شده در هسته لینوکس
Kafka	سکوی پیام‌رسانی و جریان داده توزیع‌شده
CSV	قالب متنی جدولی برای ذخیره گزارش‌ها
GC	Garbage Collection، فرایند آزادسازی حافظه در جاوا
PID	شناسه فرایند در سیستم‌عامل

فصل اول

کلیات پژوهش

۱-۱ مقدمه

سامانه‌های نرم‌افزاری امروزی معمولاً به صورت پیوسته، توزیع شده و تحت بارهای متغیر اجرا می‌شوند. در چنین محیطی، خطاهای عملکردی الزاماً به شکل توقف کامل برنامه دیده نمی‌شوند؛ گاهی افزایش زمان پاسخ، مصرف غیرعادی پردازنده، فشار حافظه، ازدحام قفل‌ها یا الگوی غیرمنتظره ورودی/خروجی فایل باعث افت کیفیت سرویس می‌شود. بنابراین، پایش عملکرد باید بتواند رخدادهای درون برنامه و نشانه‌های سطح سیستم‌عامل را به صورت جریان داده دریافت کرده و به سرعت به معیارهای قابل تحلیل تبدیل کند.

پروژه مبنای این پایان‌نامه، با نام CE-FINAL-PROJECT، یک خط لوله پایش و تشخیص ناهنجاری برای برنامه‌های جاوا است. این سامانه از Java Flight Recorder یا JFR برای مشاهده رخدادهای درون ماشین مجازی جاوا، از eBPF برای جمع‌آوری رخدادهای سطح هسته، از Kafka برای انتقال پیام، از Kafka Streams برای تجمیع پنجره‌ای، و از یک سرویس پایتونی برای تحلیل و گزارش ناهنجاری استفاده می‌کند [۳، ۲، ۵، ۴].

۲-۱ بیان مسئله

ابزارهای پایش سنتی اغلب یا به معیارهای کلی سیستم‌عامل محدود می‌شوند یا رخدادهای سطح برنامه را بدون ارتباط با زمینه اجرایی سیستم تحلیل می‌کنند. این جدایی باعث می‌شود ریشه‌یابی رخدادهایی مانند افزایش مصرف پردازنده، فشار حافظه، کندی فایل، یا ازدحام قفل دشوار شود. مسئله اصلی این پژوهش، طراحی خط لوله‌ای است که بتواند رخدادهای سطح JVM و رخدادهای سطح هسته را به صورت یک جریان یکپارچه دریافت کند، آن‌ها را در بازه‌های زمانی مشخص تجمیع کند و سپس بر اساس قواعد قابل توسعه، ناهنجاری‌های عملکردی را تشخیص دهد.

۳-۱ اهمیت و ضرورت

پایش عملکرد در محیط تولید باید چند ویژگی کلیدی داشته باشد. نخست، دریافت رخداد باید با سربار قابل قبول انجام شود تا خود ابزار پایش منبع اختلال نباشد. دوم، داده خام باید به معیارهای خلاصه و قابل مقایسه تبدیل شود. سوم، معماری باید توسعه‌پذیر باشد تا اضافه کردن نوع رخداد، معیار یا قانون تشخیص جدید نیازمند بازنویسی کل سامانه نباشد. چهارم، خروجی سامانه باید قابل گزارش‌گیری و

ارزیابی باشد. پروژه بررسی شده این ویژگی‌ها را با جداسازی اجزای دریافت، تجمیع، تحلیل و ارزیابی دنبال می‌کند.

۴-۱ اهداف پژوهش

اهداف اصلی این پایان‌نامه عبارت‌اند از:

- بررسی نیازمندی‌های یک سامانه پایش عملکرد برای برنامه‌های جاوا؛
- طراحی معماری خط لوله‌ای برای دریافت رخدادهای JFR و رخدادهای سطح هسته؛
- پیاده‌سازی تجمیع پنجره‌ای رخدادها با Kafka Streams؛
- پیاده‌سازی آشکارسازهای ناهنجاری برای پردازنده، حافظه، فایل، شبکه، قفل و رخدادهای هسته؛
- طراحی سناریوهای ارزیابی کنترل‌شده برای سنجش توان تشخیص سامانه.

۵-۱ پرسش‌های پژوهش

این پژوهش به پرسش‌های زیر پاسخ می‌دهد:

۱. چگونه می‌توان رخدادهای JFR و رخدادهای سطح هسته را در یک خط لوله مشترک پردازش کرد؟
۲. چه ساختاری برای تجمیع رخدادهای عملکردی در پنجره‌های زمانی مناسب است؟
۳. چگونه می‌توان آشکارسازهای ناهنجاری را به صورت مستقل و قابل توسعه طراحی کرد؟
۴. چه سناریوهایی برای ارزیابی رفتار سامانه پایش مناسب هستند؟

۶-۱ ساختار پایان‌نامه

در فصل دوم مفاهیم و فناوری‌های پایه مانند JFR، eBPF، Kafka و تشخیص ناهنجاری معرفی می‌شود. فصل سوم معماری سامانه و جریان داده را توضیح می‌دهد. فصل چهارم جزئیات پیاده‌سازی اجزای اصلی پروژه را بررسی می‌کند. فصل پنجم روش ارزیابی، جمع‌بندی و پیشنهادها را توسعه آینده را ارائه می‌دهد.

فصل دوم

مبانی نظری و فناوری‌های مورد استفاده

۱-۲ پایش عملکرد و مشاهده پذیری

مشاهده‌پذیری در نرم‌افزار به توانایی فهم وضعیت داخلی سامانه بر اساس خروجی‌های قابل مشاهده مانند رخدادها، معیارها و گزارش‌ها گفته می‌شود. در سامانه‌های مبتنی بر JVM، رخدادهای عملکردی می‌توانند از درون برنامه، ماشین مجازی، کتابخانه‌های زمان اجرا و سیستم‌عامل به دست آیند. ترکیب این منابع دید کامل‌تری نسبت به رفتار برنامه ایجاد می‌کند.

۲-۲ Java Flight Recorder

JFR بخشی از سکوی جاوا است که رخدادهایی درباره اجرای برنامه، مصرف پردازنده، حافظه، جمع‌آوری زباله، نخ‌ها، قفل‌ها، ورودی/خروجی و سایر رفتارهای زمان اجرا تولید می‌کند [۵]. مزیت اصلی JFR این است که برای پایش کم‌سربار طراحی شده و داده‌های آن با ساختار مشخص قابل مصرف است. در پروژه مورد بررسی، ماژول event-receiver به فرایند هدف متصل شده و رخدادهای JFR را به پیام‌های قابل ارسال در Kafka تبدیل می‌کند.

۳-۲ eBPF و رخدادهای سطح هسته

eBPF امکان اجرای برنامه‌های کوچک و کنترل‌شده در فضای هسته لینوکس را فراهم می‌کند و برای پایش شبکه، فراخوانی‌های سیستمی، امنیت و عملکرد کاربرد زیادی دارد [۴]. در این پروژه، بخش kernel-event-sender با زبان C و libbpf رخدادهای سیستمی را جمع‌آوری می‌کند و بخش kernel-event-receiver آن‌ها را دریافت و به جریان پیام تبدیل می‌کند. این مسیر باعث می‌شود سامانه علاوه بر رخدادهای سطح JVM، نشانه‌های نزدیک‌تر به سیستم‌عامل را نیز ببیند.

۴-۲ Kafka و پردازش جریانی

Kafka یک سکوی پیام‌رسانی توزیع‌شده برای انتقال جریان داده است و Kafka Streams کتابخانه‌ای برای پردازش جریانی روی داده‌های ذخیره‌شده در Kafka محسوب می‌شود [۸، ۲]. در این پروژه، رخدادهای خام در موضوع‌هایی مانند profiler.raw.events و profiler.kernel.raw.events قرار می‌گیرند. سپس ماژول event-aggregator این رخدادها را در پنجره‌های زمانی ده‌ثانیه‌ای تجمیع و خروجی را در

موضوع‌های تجمیع‌شده منتشر می‌کند.

۵-۲ تشخیص ناهنجاری

تشخیص ناهنجاری به شناسایی رفتارهایی گفته می‌شود که با الگوی معمول سامانه تفاوت معنادار دارند. در این پروژه، رویکردهای ساده اما قابل توضیح استفاده شده‌اند: قواعد آستانه‌ای برای معیارهایی که حد قابل قبول مشخص دارند، خط پایه غلتان برای مقایسه مقدار فعلی با گذشته نزدیک، و تشخیص روند افزایشی برای معیارهایی که رشد پیوسته آن‌ها می‌تواند نشانه مشکل باشد. این انتخاب‌ها برای یک پروژه مهندسی مناسب هستند، زیرا خروجی آن‌ها قابل تفسیر و پیاده‌سازی آن‌ها کم‌هزینه است.

۶-۲ جمع‌بندی

ترکیب JFR، eBPF و Kafka Streams امکان ایجاد خط لوله‌ای را فراهم می‌کند که هم رخدادهای درون ماشین مجازی و هم رخدادهای سیستم‌عامل را دریافت کند. این فصل مفاهیم لازم برای فهم طراحی سامانه را معرفی کرد و فصل بعد معماری پیشنهادی را توضیح می‌دهد.

فصل سوم

معماری سامانه

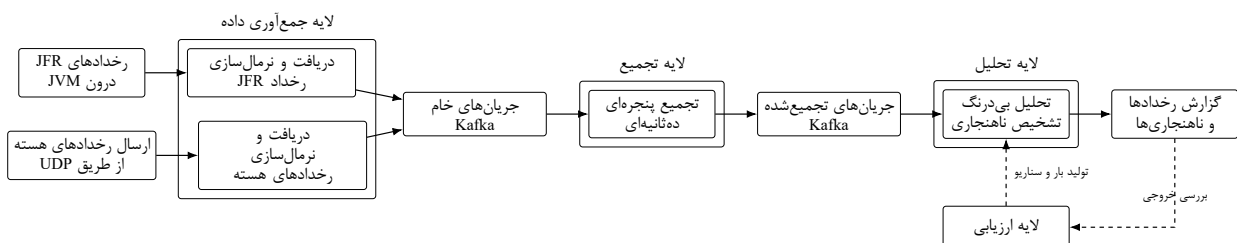
۱-۳ مقدمه

هدف اصلی سامانه، ثبت و پردازش رخدادهای مرتبط با کارایی فرایندهای JVM به گونه‌ای است که بتوان رفتار سامانه را به صورت بی‌درنگ تحلیل کرد. برای رسیدن به این هدف، معماری به صورت لایه‌ای و جریان‌محور طراحی شده است. در این معماری، رخدادها از منابع مختلف جمع‌آوری می‌شوند، به قالبی یکسان تبدیل می‌گردند، در بازه‌های زمانی کوتاه تجمیع می‌شوند و در نهایت برای تشخیص ناهنجاری مورد تحلیل قرار می‌گیرند.

نکته مهم در این فصل آن است که معماری سامانه در سطح مفهومی بررسی می‌شود. بنابراین تمرکز بر نقش هر لایه، جریان داده میان لایه‌ها و دلیل وجود هر بخش است، نه جزئیات پیاده‌سازی داخلی اجزا.

۲-۳ نمای کلی معماری

سامانه از چند لایه منطقی تشکیل شده است: لایه جمع‌آوری داده، لایه ذخیره‌سازی و انتقال جریان داده، لایه تجمیع، لایه تحلیل و لایه ارزیابی. لایه جمع‌آوری، رخدادها را از دو مسیر مستقل دریافت می‌کند. مسیر اول مربوط به رخدادهای JFR است که رفتار درون JVM را نشان می‌دهند. مسیر دوم مربوط به رخدادهای سطح هسته است که از بیرون فرایند و در سطح سیستم‌عامل مشاهده می‌شوند. پس از جمع‌آوری، هر دو مسیر داده به قالب مشترک تبدیل می‌شوند و در Kafka ذخیره می‌گردند. سپس لایه تجمیع، داده‌های خام را در پنجره‌های زمانی ده‌ثانیه‌ای خلاصه می‌کند و نتیجه را دوباره در Kafka قرار می‌دهد. در نهایت، لایه تحلیل داده‌های تجمیع‌شده را دریافت کرده و بررسی می‌کند که آیا نشانه‌ای از ناهنجاری عملکردی وجود دارد یا خیر.



شکل ۱-۳: نمای کلی معماری لایه‌ای سامانه

شکل ۱-۳ نمای کلی این معماری را نشان می‌دهد. همان‌طور که در شکل دیده می‌شود، مسیرهای

JFR و رخدادهای هسته در ابتدا جدا هستند، اما پس از نرمال سازی وارد مسیر مشترک ذخیره سازی، تجمیع و تحلیل می شوند.

۳-۳ مسیرهای داده در سامانه

در طراحی سامانه دو مسیر داده اصلی وجود دارد. این دو مسیر از نظر منبع تولید رخداد متفاوت اند، اما پس از نرمال سازی در یک مدل پردازشی مشترک قرار می گیرند.

۱-۳-۳ مسیر رخدادهای JFR

مسیر اول از رخدادهای JFR شروع می شود. این رخدادهای اطلاعاتی درباره رفتار درون JVM تولید می کنند؛ برای نمونه مصرف پردازنده، وضعیت حافظه، رخدادهای ورودی/خروجی فایل، ارتباطات شبکه، نخ ها و قفل ها. مولفه دریافت کننده JFR این رخدادهای را از فرایند هدف دریافت می کند و پیش از ذخیره سازی، آن ها را نرمال سازی می کند.

منظور از نرمال سازی در این مسیر، تبدیل رخدادهای خام JFR به ساختاری یکنواخت است؛ ساختاری که در آن منبع رخداد، نوع رخداد، دسته رخداد، زمان، مشخصات فرایند، اطلاعات نخ و داده های اختصاصی رخداد مشخص باشد. پس از این تبدیل، رخدادهای به عنوان داده خام در Kafka ذخیره می شوند تا لایه های بعدی بتوانند بدون وابستگی مستقیم به منبع JFR آن ها را پردازش کنند.

۲-۳-۳ مسیر رخدادهای هسته

مسیر دوم مربوط به رخدادهای سطح هسته یا kernel events است. این رخدادهای دیدی بیرونی تر نسبت به رفتار فرایند فراهم می کنند و برای مشاهده مواردی مانند رخدادهای فایل، شبکه و حافظه در سطح سیستم عامل مفید هستند. در این مسیر، رخدادهای هسته از طریق UDP به مولفه دریافت کننده رخدادهای هسته ارسال می شوند.

مولفه دریافت کننده هسته پس از دریافت پیام ها، آن ها را به مدل مشترک سامانه تبدیل می کند. در این مرحله، فیلدهایی مانند نوع رخداد، شناسه فرایند، شناسه نخ، زمان رخداد و دسته عملکردی استخراج یا تکمیل می شوند. سپس رخدادهای نرمال شده در مسیر خام Kafka ذخیره می شوند. به این ترتیب، هرچند منبع رخدادهای هسته با رخدادهای JFR متفاوت است، اما خروجی این لایه از نظر ساختاری با مسیر اول هم راستا می شود.

۴-۳ لایه جمع‌آوری داده

لایه جمع‌آوری داده مسئول تبدیل داده‌های خام ناهمگون به رخدادهای ساخت‌یافته و قابل پردازش است. این لایه دو وظیفه اصلی دارد: نخست، اتصال به منبع رخداد و دریافت داده؛ دوم، نرمال‌سازی رخدادهای و انتشار آن‌ها در جریان داده.

جداسازی مسیر JFR و مسیر رخدادهای هسته در این لایه یک تصمیم معماری مهم است. دلیل این جداسازی آن است که هر منبع داده ویژگی‌ها، نرخ تولید، روش اتصال و سطح مشاهده متفاوتی دارد. با این حال، هر دو مسیر پس از نرمال‌سازی به مدل مشترک سامانه می‌رسند. این کار باعث می‌شود لایه‌های بعدی، یعنی تجمیع و تحلیل، نیازی به شناخت جزئیات منبع اولیه نداشته باشند. در این لایه داده هنوز در سطح رخداد خام قرار دارد. بنابراین حجم داده می‌تواند زیاد باشد و نگهداری بلندمدت آن هزینه ذخیره‌سازی بالایی ایجاد کند. به همین دلیل، معماری سامانه یک لایه تجمیع جداگانه در ادامه مسیر قرار می‌دهد تا داده خام به معیارهای خلاصه‌تر تبدیل شود.

۵-۳ لایه انتقال و ذخیره‌سازی جریان داده

Kafka در این معماری نقش ستون فقرات انتقال داده را دارد. هر لایه، خروجی خود را در یک جریان مشخص منتشر می‌کند و لایه بعدی آن جریان را مصرف می‌کند. این طراحی چند مزیت دارد. نخست، اجزا از نظر اجرایی از یکدیگر جدا می‌شوند و توقف یا کندی یک بخش الزاماً باعث توقف کامل سامانه نمی‌شود. دوم، امکان کنترل نگهداری داده در هر مرحله وجود دارد. سوم، افزودن مصرف‌کننده‌های جدید، مانند داشبورد یا گزارش‌گیر مستقل، بدون تغییر در لایه جمع‌آوری امکان‌پذیر می‌شود.

در سطح معماری، دو نوع جریان در Kafka قابل تشخیص است: جریان‌های خام و جریان‌های تجمیع‌شده. جریان خام شامل رخدادهای نرمال‌شده‌ای است که مستقیماً از منابع JFR یا هسته به دست آمده‌اند. جریان تجمیع‌شده شامل خروجی پنجره‌های زمانی است و برای تحلیل و گزارش‌گیری مناسب‌تر است.

۶-۳ لایه تجمیع

لایه تجمیع، رخدادهای خام را به معیارهای قابل تحلیل تبدیل می‌کند. در این سامانه، تجمیع در پنجره‌های زمانی ده‌ثانیه‌ای انجام می‌شود. برای هر پنجره، رخدادهای بر اساس کلیدهایی مانند شناسه

فرایند، منبع رخداد، دسته رخداد و نوع رخداد گروه‌بندی می‌شوند و سپس معیارهایی مانند تعداد رخداد، مجموع مدت، میانگین مدت، بیشینه مدت و معیارهای اختصاصی هر دسته محاسبه می‌گردد. وجود این لایه از چند جهت اهمیت دارد. نخست، تحلیل ناهنجاری معمولاً بر اساس روندها و معیارهای خلاصه‌شده معنادارتر از تحلیل تک‌تک رخدادهای خام است. دوم، تجمیع باعث کاهش حجم داده‌ای می‌شود که باید برای مدت طولانی‌تر نگهداری شود. در نتیجه، لازم نیست برای لایه جمع‌آوری داده، زمان نگهداری یا retention زیادی در نظر گرفته شود. داده خام می‌تواند با زمان نگهداری کوتاه‌تر ذخیره شود و داده تجمیع‌شده، که حجم کم‌تر و ارزش تحلیلی بیش‌تری دارد، برای تحلیل و گزارش‌گیری حفظ گردد. این تصمیم باعث کاهش مصرف دیسک و ساده‌تر شدن مدیریت داده می‌شود. سوم، لایه تجمیع مرزی میان مشاهده رخداد و تحلیل رفتاری ایجاد می‌کند. لایه جمع‌آوری صرفاً رخداد را ثبت می‌کند، اما لایه تجمیع آن رخدادها را به شاخص‌هایی تبدیل می‌کند که برای تصمیم‌گیری قابل استفاده‌اند.

۷-۳ لایه تحلیل

لایه تحلیل، مصرف‌کننده داده‌های تجمیع‌شده است. این لایه خروجی پنجره‌های زمانی را دریافت می‌کند و بررسی می‌کند که آیا معیارهای مشاهده‌شده نشانه‌ای از رفتار غیرعادی دارند یا خیر. از آنجا که ورودی این لایه داده خام نیست، تحلیل‌گر با معیارهایی کار می‌کند که از قبل خلاصه، دسته‌بندی و زمان‌بندی شده‌اند.

در این معماری، تحلیل ناهنجاری می‌تواند برای دسته‌های مختلف رخداد به صورت جداگانه انجام شود؛ برای نمونه پردازنده، حافظه، ورودی/خروجی فایل، شبکه، قفل‌ها و رخدادهای هسته. این جداسازی باعث می‌شود منطق تشخیص برای هر دسته متناسب با ماهیت همان دسته تعریف شود. برای مثال، معیار مناسب برای فشار پردازنده با معیار مناسب برای ازدحام قفل یا فشار حافظه یکسان نیست. خروجی لایه تحلیل، گزارش رخدادهای تجمیع‌شده و گزارش ناهنجاری‌ها است. گزارش ناهنجاری نشان می‌دهد در چه بازه زمانی، برای چه دسته‌ای از رخدادها و بر اساس کدام معیار، رفتار غیرعادی مشاهده شده است.

۳-۸ لایه ارزیابی

در کنار لایه‌های اصلی سامانه، یک لایه ارزیابی نیز وجود دارد. این لایه بخشی از مسیر عادی پایش نیست، بلکه برای سنجش توان سامانه استفاده می‌شود. نقش آن ایجاد سناریوهای کنترل‌شده و بررسی این موضوع است که آیا لایه تحلیل می‌تواند ناهنجاری مورد انتظار را تشخیص دهد یا خیر. لایه ارزیابی سناریوهایی مانند افزایش مصرف پردازنده، فشار حافظه و جمع‌آوری زباله، ورودی/خروجی سنگین فایل، ترافیک شبکه، رقابت روی قفل‌ها و سناریوهای وابسته به رخدادهای هسته را ایجاد می‌کند. سپس خروجی تحلیل‌گر بررسی می‌شود و گزارشی شامل وضعیت تشخیص، معیار تشخیص‌داده‌شده و تأخیر تشخیص تولید می‌گردد.

۳-۹ مدل داده مشترک

یکی از نکات کلیدی معماری، تعریف مدل داده مشترک پس از لایه جمع‌آوری است. هر رخداد، صرف نظر از این که از JFR آمده باشد یا از مسیر هسته، باید اطلاعات پایه‌ای مشابهی داشته باشد. این اطلاعات شامل منبع رخداد، نوع رخداد، دسته رخداد، زمان، شناسه فرایند و داده‌های تکمیلی است. وجود مدل مشترک باعث می‌شود لایه تجمیع بتواند هر دو مسیر داده را با منطق یکسان پردازش کند. همچنین لایه تحلیل می‌تواند بر اساس دسته و معیار تصمیم بگیرد، نه بر اساس جزئیات منبع تولید رخداد. این انتزاع یکی از دلایل توسعه‌پذیری معماری است؛ زیرا با اضافه شدن منبع داده جدید، کافی است خروجی آن منبع به مدل مشترک تبدیل شود.

۳-۱۰ دلایل انتخاب معماری لایه‌ای

انتخاب معماری لایه‌ای برای این سامانه چند دلیل اصلی دارد:

- **جداسازی مسئولیت‌ها:** هر لایه یک وظیفه مشخص دارد؛ جمع‌آوری، انتقال، تجمیع، تحلیل یا ارزیابی.
- **کاهش وابستگی اجزا:** ارتباط اجزا از طریق جریان‌های Kafka انجام می‌شود و هر بخش می‌تواند مستقل‌تر توسعه یا اجرا شود.

- **مدیریت بهتر حجم داده:** داده خام با نگهداری کوتاه‌تر ذخیره می‌شود و داده تجمیع‌شده برای تحلیل و گزارش حفظ می‌گردد.
- **پشتیبانی از چند منبع مشاهده:** معماری می‌تواند هم رخدادهای درون JVM و هم رخدادهای سطح هسته را در یک مدل تحلیلی مشترک قرار دهد.
- **امکان توسعه آینده:** افزودن منبع داده، معیار تجمیع یا قانون تحلیل جدید بدون تغییر بنیادین در کل سامانه امکان‌پذیر است.

۱۱-۳ جمع‌بندی

در این فصل معماری سامانه از دید لایه‌ای بررسی شد. سامانه دو مسیر داده اصلی دارد: مسیر رخدادهای JFR و مسیر رخدادهای هسته. هر دو مسیر در لایه جمع‌آوری نرمال‌سازی شده و در Kafka ذخیره می‌شوند. سپس لایه تجمیع، داده‌های خام را در پنجره‌های ده‌ثانیه‌ای به معیارهای خلاصه تبدیل می‌کند. لایه تحلیل این معیارها را برای تشخیص ناهنجاری مصرف می‌کند و لایه ارزیابی نیز با ایجاد سناریوهای کنترل‌شده، توان سامانه را می‌سنجد. این معماری با جداسازی مسئولیت‌ها و استفاده از مدل داده مشترک، امکان تحلیل بی‌درنگ و توسعه‌پذیر رخدادهای کارایی فرایندهای JVM را فراهم می‌کند.

فصل چهارم

پیاده‌سازی سامانه

۱-۴ ساختار مخزن

مخزن پروژه یک پروژه چندماژوله است. فایل اصلی pom.xml ماژول‌های event-receiver، kernel-event-receiver، event-aggregator و profiler-evaluation را تعریف می‌کند و نسخه جاوا را ۱۷ قرار می‌دهد. بخش event-analytics با پایتون پیاده‌سازی شده و وابستگی‌های آن در فایل requirements.txt آمده است. بخش kernel-event-sender نیز شامل کد eBPF، بارگذار C و Makefile است.

۲-۴ دریافت رخدادهای JFR

ورودی اصلی رخدادهای سطح برنامه در کلاس aut.ce.App از ماژول event-receiver آغاز می‌شود. این کلاس نام ماشین مجازی هدف را از متغیر محیطی TARGET_VM_NAME می‌خواند و در صورت نبود مقدار، نام JfrEventGenerator را در نظر می‌گیرد. سپس JfrRemoteListener و JfrRemoteEventStreamer اجرا می‌شوند و نخ KafkaWriter پیام‌ها را به Kafka ارسال می‌کند. برای استخراج فیلدهای تخصصی، الگوی غنی‌ساز رخداد استفاده شده است. هر غنی‌ساز بخشی از بار رخداد را می‌سازد و خروجی نهایی به صورت پیام ساخت‌یافته ارسال می‌شود. این روش باعث می‌شود اضافه کردن یک نوع رخداد جدید با افزودن یک غنی‌ساز جدید انجام شود.

۳-۴ دریافت رخدادهای هسته

بخش kernel-event-sender شامل برنامه eBPF و بارگذار کاربرفضا است. برنامه eBPF رخدادهای مرتبط را از هسته دریافت می‌کند و بارگذار آن‌ها را به مقصد شبکه ارسال می‌کند. ماژول kernel-event-receiver این رخدادها را دریافت، بافر و در Kafka منتشر می‌کند. این مسیر برای سناریوهایی مفید است که نشانه اصلی آن‌ها در سطح سیستم‌عامل دیده می‌شود.

۴-۴ تجمیع رخدادها

کلاس EventAggregatorApp قلب ماژول تجمیع است. این کلاس دو توپولوژی مشابه می‌سازد: یکی برای رخدادهای JFR و دیگری برای رخدادهای هسته. رخدادها ابتدا فیلتر می‌شوند تا پیام‌های نامعتبر حذف شوند. سپس بر اساس کلید تجمیع گروه‌بندی شده و در پنجره‌های زمانی پردازش می‌شوند.

خروجی هر پنجره شامل زمان شروع و پایان، مدت پنجره، کلید و معیارهای محاسبه شده است. راهبردهای تجمیع در بسته strategy قرار دارند. این راهبردها تفاوت رخدادهای پردازنده، حافظه، فایل، شبکه و جمع‌آوری زباله را از منطق عمومی Kafka Streams جدا می‌کنند. در نتیجه، تغییر روش محاسبه یک معیار نیازمند تغییر توپولوژی اصلی نیست.

۴-۵ تحلیل و گزارش‌گیری

در ماژول event-analytics، فایل main.py پیکربندی را می‌خواند، مصرف‌کننده Kafka را می‌سازد، آشکارسازها را مقداردهی می‌کند و پیام‌های تجمیع‌شده را پردازش می‌کند. آشکارسازهای موجود شامل پردازنده، حافظه، ورودی/خروجی فایل، شبکه، قفل و هسته هستند. خروجی‌ها توسط CsvReportWriter در قالب گزارش رخدادهای تجمیع‌شده و گزارش ناهنجاری نوشته می‌شوند.

۴-۶ پایش خود سامانه

پروژه برای مشاهده وضعیت اجزای خود نیز از Prometheus استفاده می‌کند. در ماژول‌های جاوا، کلاس‌های MetricsHandler معیارهایی مانند مقدارهای تجمیع‌شده را به‌روزرسانی می‌کنند. پوشه monitoring شامل پیکربندی لازم برای اجرای Prometheus است [۶].

۴-۷ جمع‌بندی

پیاده‌سازی پروژه بر اصل جداسازی مسئولیت‌ها تکیه دارد. دریافت‌کننده‌ها مسئول تبدیل رخداد خام به پیام، تجمیع‌کننده مسئول تبدیل پیام‌ها به معیارهای پنجره‌ای، و تحلیل‌گر مسئول تصمیم‌گیری درباره ناهنجاری‌ها است.

فصل پنجم

ارزیابی، نتیجه‌گیری و کارهای آینده

۱-۵ روش ارزیابی

برای ارزیابی سامانه، ماژول profiler-evaluation سناریوهای کنترل‌شده‌ای را اجرا می‌کند. این ماژول یک فرایند جاوا راه‌اندازی می‌کند، شناسه فرایند را چاپ می‌کند و پس از یک تأخیر اولیه، سناریوها را اجرا می‌کند تا دریافت‌کننده‌های JFR و هسته فرصت اتصال داشته باشند. خروجی ارزیابی شامل فایل‌های experiment_results.csv، experiment_metrics.json و evaluation_report.md است.

۲-۵ سناریوهای آزمایش

سناریوهای پیاده‌سازی‌شده در پروژه عبارت‌اند از:

- CpuSpikeScenario برای ایجاد افزایش ناگهانی مصرف پردازنده؛
- MemoryGcPressureScenario برای ایجاد فشار حافظه و تحریک جمع‌آوری زباله؛
- FileIoBurstScenario برای تولید الگوی شدید ورودی/خروجی فایل؛
- LockContentionScenario برای ایجاد رقابت روی قفل‌ها؛
- NetworkBurstScenario برای ایجاد ترافیک شبکه؛
- MmapFileScanScenario برای بررسی رفتار دسترسی حافظه‌نگاشته به فایل.

هر سناریو یک رفتار هدفمند ایجاد می‌کند و سپس گزارش تحلیلی بررسی می‌شود تا مشخص شود آیا ناهنجاری مورد انتظار تشخیص داده شده است یا خیر. این روش برای پروژه حاضر مناسب است، زیرا امکان کنترل زمان شروع، مدت اجرا و نوع فشار ایجادشده را فراهم می‌کند.

۳-۵ معیارهای ارزیابی

دو معیار اصلی برای ارزیابی در نظر گرفته می‌شود. معیار نخست موفقیت تشخیص است؛ یعنی آیا سامانه برای سناریوی اجراشده، ناهنجاری مناسب را گزارش کرده است یا نه. معیار دوم تأخیر تشخیص است؛ یعنی فاصله زمانی بین شروع سناریو و ثبت ناهنجاری در گزارش. این دو معیار در کنار هم نشان می‌دهند سامانه هم از نظر دقت عملی و هم از نظر سرعت واکنش چه وضعیتی دارد.

۴-۵ بحث

طراحی خط لوله‌ای پروژه چند مزیت مهم دارد. نخست، هر جزء به صورت مستقل اجرا و عیب‌یابی می‌شود. دوم، استفاده از Kafka امکان جداسازی نرخ تولید و مصرف پیام را فراهم می‌کند. سوم، جمع‌آوری پنجره‌ای حجم داده خام را کاهش داده و تحلیل را ساده‌تر می‌کند. چهارم، ترکیب رخدادهای JFR و eBPF دید گسترده‌تری نسبت به رفتار برنامه ایجاد می‌کند.

در مقابل، سامانه محدودیت‌هایی نیز دارد. کیفیت تشخیص به انتخاب آستانه‌ها و طول پنجره وابسته است. همچنین اجرای بخش eBPF به محیط لینوکس و دسترسی‌های لازم نیاز دارد. از سوی دیگر، بسیاری از آزمون‌های موجود در پروژه حالت اسکلت اولیه دارند و برای استفاده پژوهشی کامل، لازم است پوشش آزمون واحد و آزمون یکپارچه افزایش یابد.

۵-۵ نتیجه‌گیری

در این پایان‌نامه، سامانه‌ای برای پایش عملکرد و تشخیص ناهنجاری در برنامه‌های جاوا بررسی، طراحی و مستندسازی شد. سامانه با استفاده از JFR رخدادهای سطح ماشین مجازی را دریافت می‌کند، با eBPF امکان مشاهده رخدادهای سطح هسته را فراهم می‌سازد، با Kafka Streams داده‌ها را در پنجره‌های زمانی جمع‌آوری می‌کند و با سرویس تحلیلی پایتونی ناهنجاری‌ها را گزارش می‌دهد. نتیجه اصلی این پژوهش آن است که ترکیب مشاهده‌پذیری سطح برنامه و سطح سیستم‌عامل در یک معماری جریانی، امکان تحلیل دقیق‌تر و توسعه‌پذیرتر رفتار عملکردی را فراهم می‌کند.

۶-۵ کارهای آینده

برای توسعه آینده پروژه، موارد زیر پیشنهاد می‌شود:

- افزودن داشبورد تصویری برای نمایش زنده معیارها و ناهنجاری‌ها؛
- استفاده از روش‌های یادگیری ماشین برای تنظیم پویا و داده‌محور آستانه‌ها؛
- افزایش پوشش آزمون‌های واحد و یکپارچه برای ماژول‌های جاوا و پایتون؛
- افزودن همبستگی بین رخدادهای JFR و رخدادهای هسته بر اساس زمان و شناسه فرایند؛

- پشتیبانی از استقرار کانتینری کامل برای اجرای ساده‌تر کل خط لوله؛
- افزودن معیارهای دقت، فراخوانی و نرخ هشدار کاذب به گزارش ارزیابی.

کتاب نامه

- [1] Apache Software Foundation. Apache kafka documentation. <https://kafka.apache.org/documentation/>, 2026. Accessed 2026-06-23.
- [2] Apache Software Foundation. Kafka streams documentation. <https://kafka.apache.org/documentation/streams/>, 2026. Accessed 2026-06-23.
- [3] CaOmDiEn. Ce-final-project: Jvm profiler and anomaly detection pipeline. <https://github.com/CaOmDiEn/CE-FINAL-PROJECT>, 2026. Commit b23e1b4bc1224cb0ea922dff86189ed8c3d5f437, accessed 2026-06-23.
- [4] eBPF Foundation. ebpf documentation. <https://ebpf.io/what-is-ebpf/>, 2026. Accessed 2026-06-23.
- [5] Oracle. Java flight recorder runtime guide. <https://docs.oracle.com/javacomponents/jmc-5-4/jfr-runtime-guide/about.htm>, 2026. Accessed 2026-06-23.
- [6] Prometheus Authors. Prometheus documentation. <https://prometheus.io/docs/introduction/overview/>, 2026. Accessed 2026-06-23.

پیوست اول

راهنمای اجرای پروژه

این پیوست خلاصه‌ای از روش اجرای خط لوله پروژه را ارائه می‌کند. دستورهای زیر بر اساس ساختار مخزن CE-FINAL-PROJECT نوشته شده‌اند.

آ-۱ ساخت ماژول‌های جاوا

```
mvn package -DskipTests
```

آ-۲ اجرای اجزای اصلی

ابتدا Kafka و سرویس‌های وابسته اجرا می‌شوند. سپس دریافت‌کننده رخدادهای JFR، دریافت‌کننده رخدادهای هسته، تجمیع‌کننده و تحلیل‌گر اجرا می‌شوند. اسکریپت `scripts/run-profiler-pipelines.sh` برای هماهنگ کردن این مراحل در مخزن پروژه قرار دارد.

آ-۳ اجرای ارزیابی

برای اجرای ماژول ارزیابی:

```
mvn -pl profiler-evaluation -am package -DskipTests
```

```
java -cp profiler-evaluation/target/classes aut.ce.evaluation.App
```

برنامه در شروع اجرا شناسه فرایند را چاپ می‌کند. برای بررسی رخدادهای هسته، فرستنده eBPF با همان شناسه فرایند ساخته و اجرا می‌شود:

```
cd kernel-event-sender
```

```
make clean
```

```
make TARGET_PID=12345
```

```
sudo ./build/loader
```

آ-۴ خروجی‌ها

خروجی‌های ارزیابی در قالب فایل‌های زیر تولید می‌شوند:

experiment_results.csv

experiment_metrics.json

evaluation_report.md

گزارش نهایی شامل وضعیت موفقیت تشخیص در هر سناریو و تأخیر تشخیص است.

واژه‌نامه‌ی فارسی به انگلیسی

Detection تشخیص	آ
خ	
Rolling Baseline خط پایه غلتان	آشکارسازی ناهنجاری . Anomaly Detection
د	آستانه Threshold
	ا
Event Receiver دریافت‌کننده رخداد	ارزیابی Evaluation
ر	
Event رخداد	ازدحام قفل Lock Contention
	ب
Kernel Event رخداد هسته	
ز	بازه زمانی Time Window
Runtime زمان اجرا	بافر Buffer
س	ت
Overhead سربار	تجمیع Aggregation

Observability نظارت‌پذیری	Experiment Scenario . . . سناریوی آزمایش
و	ش
File Input/Output ورودی/خروجی فایل	Process Identifier شناسه فرایند
	ص
	Message Queue صف پیام
	ف
	Memory Pressure فشار حافظه
	Target Process فرایند هدف
	ک
	Aggregation Key کلید تجمیع
	گی
	Anomaly Report گزارش ناهنجاری
	م
	Java Virtual Machine . . ماشین مجازی جاوا
	Metric معیار
	Topic موضوع
	ن
	Anomaly ناهنجاری

واژه‌نامه‌ی انگلیسی به فارسی

A	Experiment Scenario . . . سناریوی آزمایش
Aggregation تجمیع	F
Aggregation Key کلید تجمیع	File Input/Output . . . ورودی/خروجی فایل
Anomaly ناهنجاری	J
Anomaly Detection . آشکارسازی ناهنجاری	Java Virtual Machine . . ماشین مجازی جاوا
Anomaly Report گزارش ناهنجاری	K
B	Kernel Event رخداد هسته
Buffer بافر	L
D	Lock Contention ازدحام قفل
Detection تشخیص	M
E	Memory Pressure فشار حافظه
Evaluation ارزیابی	Message Queue صف پیام
Event رخداد	Metric معیار
Event Receiver دریافت‌کننده رخداد	O
	Observability نظارت‌پذیری

Overhead	سربار	Runtime	زمان اجرا
P		T	
Process Identifier	شناسه فرایند	Target Process	فرایند هدف
R		Threshold	آستانه
Rolling Baseline	خط پایه غلتان	Time Window	بازه زمانی
		Topic	موضوع

Abstract

Modern software systems require continuous observability mechanisms that can capture performance symptoms with low overhead and transform them into actionable reports. This thesis presents the design and implementation of a streaming profiler pipeline for Java applications. The system combines Java Flight Recorder events, kernel-level events collected through eBPF, Apache Kafka for event transport, Kafka Streams for windowed aggregation, and a Python analytics service for anomaly detection.

The proposed architecture separates event collection, aggregation, detection, and evaluation into independent components. JFR and kernel receivers normalize raw observations into structured Kafka messages. The aggregation service groups events in configurable time windows and computes metrics for CPU usage, memory pressure, file I/O, network traffic, lock contention, garbage collection, and kernel counters. The analytics service applies threshold-based, rolling-baseline, and trend-based detectors and writes aggregate and anomaly reports in CSV format. An evaluation module executes controlled workload scenarios, including CPU spikes, memory and GC pressure, file I/O bursts, lock contention, network bursts, and memory-mapped file scans, then checks whether the pipeline reports the expected anomalies. The implementation demonstrates that a modular stream-processing design can combine JVM-level and operating-system-level telemetry while keeping each processing stage independently deployable and extensible.

Keywords: Performance monitoring, anomaly detection, Java Flight Recorder, eBPF, Kafka Streams, observability.



**Amirkabir University of Technology
(Tehran Polytechnic)**

Department of Computer Engineering

M. Sc. Thesis

**Design and Implementation of a
Performance Monitoring and Anomaly
Detection Pipeline for Java Applications
Using JFR, eBPF, and Kafka**

By

Student Name Student Surname

Supervisor

Dr. Supervisor Name

Advisor

Dr. Advisor Name

Month & Year